

Week 6: JavaScript 2

ผศ. ดร. เกี๊จแกว ธเนศวร

kejkaew.tha@kmutt.ac.th

ตารางเรียน

สัปดาห์	วันที่	หัวข้อ/รายละเอียด	Assignment (ผู้ตรวจ)	ส่ง Assignment
1	4/8/2569	Introduction to Web Development & HTML5	Assignment 1 (KT,KN)	1
2	11/8/2569	CSS3 Fundamentals	Assignment 2 (KT, KN)	1,2
3	18/8/2569	Advanced CSS & UI Layouts	Assignment 3 (KT)	1,2,3
4	25/8/2569	Responsive Web Design	Assignment 4 (KT, KN)	2,3,4
5	1/9/2569	JavaScript 1: Variables, Data Types, DOM & Event Handling	Assignment 5 (KT, KN)	3,4,5
6	8/9/2569	JavaScript 2: Arrays, Objects & Logic Control	Assignment 6 (KT)	4,5,6
7	15/9/2569	JavaScript 3: DOM Manipulation	Assignment 7 (KT, KN)	5,6,7
8	22/9/2569	Asynchronous JS, AJAX & JSON Data Flow	Assignment 8 (KT, KN)	6,7,8
9	29/9/2569	Introduction to Node.js & Express Basics	Assignment 9 (KT)	7,8,9
10	6/10/2569	Backend Architecture & RESTful API (CRUD)	Assignment 10 (KT, KN)	8,9,10
	13/10/2569	หยุด		
	20/10/2569	หยุด		
	27/10/2569	หยุด		
11	3/11/2569	Microservices Architecture Concept & Hands-on	Assignment 11 (KT, KN)	9,10,11
12	10/11/2569	Database Integration (MySQL / Firebase)	Assignment 12 (KT, KN)	10,11,12
13	17/11/2569			11,12
14	24/11/2569	ส่งโปรเจก (9:00-16:00)	KT	12
15	12/2569	สอบปลายภาค ในตาราง		

วันนี้เราจะเรียนเรื่อง...

- Math object
- Loops
- Array
- Unobtrusive JavaScript
- Form
- GitHub

Math object

- เมธอด (Method) : abs, ceil, cos, floor, log, max, min, pow, random, round, sin, sqrt, tan
- พร็อพเพอร์ตี้ (Property): E, PI

main.js

```
10 console.log("Math.PI:", Math.PI);  
11 console.log("Math.round(4.7):", Math.round(4.7));  
12 const powerVal = Math.pow(8, 2);  
13 console.log("Math.pow(8, 2):", powerVal);
```

for loop (same as Java)

```
for (initialization; condition; update) {  
    statements;  
}
```

main.js

```
22 let sumFor = 0;  
23 for (let i = 0; i < 100; i++) {  
24     sumFor += i;  
25 }  
26 console.log("Sum with for loop:", sumFor);
```

while loops (same as Java)

```
while (condition) {  
    statements;  
}
```

```
main.js  
29 let sumWhile = 0;  
30 let j = 0;  
31 while (j < 100) {  
32     sumWhile += j;  
33     j++;  
34 }  
35 console.log("Sum with while loop:", sumWhile);
```

แบบฝึกหัด

1. คำนวณผลรวมตั้งแต่ 0 ถึง 99 ที่หารด้วย 2 หรือ 5 ลงตัวโดยใช้ while loop
2. คำนวณผลรวมตั้งแต่ 0 ถึง 99 ที่หารด้วย 3 และ 5 ลงตัวโดยใช้ for loop

Arrays

```
var array_name = [item1, item2, ...];
```

- พร็อพเพอร์ตี้ length แสดงจำนวนสมาชิก และเพิ่มขึ้นเมื่อเพิ่มสมาชิกต่อท้าย
- ดัชนีของอาร์เรย์เริ่มต้นที่ 0
- ตัวอย่างการประกาศและแก้ไขสมาชิกในอาร์เรย์

Creating an Array

สร้างอาร์เรย์และอ่านสมาชิก

main.js

```
42 const names = [5, 8, 9, 7, 4, 10];  
43 console.log("Element at index 5:", names[5]);
```

แก้ไขสมาชิกในอาร์เรย์

main.js

```
44 names[5] = 6;  
45 console.log("Updated names:", names);
```

Array methods

- อาร์เรย์ใช้แทนโครงสร้างข้อมูลได้หลายแบบ เช่น ลิสต์ คิว และสแต็ก
- เมธอด: concat, join, pop, push, reverse, shift, slice, sort, splice, toString, unshift
- push และ pop ใช้เพิ่มและลบสมาชิกตัวสุดท้ายของอาร์เรย์
- unshift และ shift ใช้เพิ่มและลบสมาชิกตัวแรกของอาร์เรย์
- shift และ pop คืนค่าของสมาชิกที่ถูกลบออกมา

ตัวอย่าง

main.js

```
48 const fruits = ["Banana", "Orange", "Apple", "Mango"];
49 console.log("Initial fruits:", fruits);
50
51 fruits.pop(); // เอาตัวท้ายออก
52 console.log("After pop():", fruits);
53
54 fruits.push("Mango"); // ใส่ตัวท้ายเพิ่ม
55 console.log("After push('Mango'):", fruits);
56
57 fruits.shift(); // เอาตัวหน้าออก
58 console.log("After shift():", fruits);
59
60 fruits.unshift("Banana"); // ใส่ตัวหน้าเพิ่ม
61 console.log("After unshift('Banana'):", fruits);
62
63 fruits.sort(); // เรียงลำดับตัวอักษร
64 console.log("After sort():", fruits);
```

["Banana", "Orange",
"Apple"]

["Banana", "Orange",
"Apple", "Mango"]

["Orange", "Apple",
"Mango"]

["Banana", "Orange",
"Apple", "Mango"]

["Apple", "Banana",
"Mango", "Orange"]

Array as an object

- ใน JavaScript อาร์เรย์ใช้ดัชนีเป็นตัวเลข
- ออบเจกต์ใช้ชื่อพร็อพเพอร์ตี้ในการเข้าถึงข้อมูล
- การเขียนออบเจกต์โดยตรง สามารถใส่พร็อพเพอร์ตี้และเมธอดได้
- ตัวอย่าง

```
main.js
72 const person = {
73   firstName: "John",
74   lastName: "Doe",
75   age: 46,
76   fullName: function() {
77     return this.firstName + " " + this.lastName;
78   }
79 };
```

ตัวอย่าง 2

main.js

```
72 const person = {  
73   firstName: "John",  
74   lastName: "Doe",  
75   age: 46,  
76   fullName: function() {  
77     return this.firstName + " " + this.lastName;  
78   }  
79 };  
80  
81 console.log("Person Object:", person);
```

Accessing Object Properties

- เราสามารถเข้าถึงพรีอเพอร์ตีของออบเจกต์ได้ 2 แบบ
- `objectName.propertyName`
- `objectName["propertyName"]`

main.js

```
81 console.log("Person Object:", person);  
82 console.log("Dot notation:", person.lastName);  
83 console.log("Bracket notation:", person["lastName"]);
```

Object methods

- ออบเจ็กต์สามารถมีเมธอดได้
- เมธอดคือการทำงานที่กำหนดไว้ภายในออบเจ็กต์
- เราสร้างเมธอดได้โดยใช้ฟังก์ชัน
- ตัวอย่างเช่น

main.js

```
72 const person = {  
73   firstName: "John",  
74   lastName: "Doe",  
75   age: 46,  
76   fullName: function() {  
77     return this.firstName + " " + this.lastName;  
78   }  
79 };  
84 console.log("Method call:", person.fullName());
```

คีย์เวิร์ด `this` ใช้อ้างถึงออบเจกต์ที่เรียกใช้เมธอดนั้น
ในตัวอย่างนี้ `this` อ้างถึงออบเจกต์ `person`

For/of (ES6)

- คำสั่ง for/of ใช้วนผ่านสมาชิกของอาร์เรย์หรือออบเจกต์ที่วนซ้ำได้ โดยไม่ต้องใช้ดัชนีหรือคีย์
- ตัวอย่างเช่น

```
main.js
91 const cars = ["BMW", "Volvo", "Mini"];
92 let carText = "";
93
94 for (const car of cars) {
95     carText += car + " ";
96 }
97 console.log("Cars:", carText.trim());
```

การสร้าง Array of Objects

- เราสามารถสร้างชุดข้อมูลที่มี objects หลายตัวได้

```
105 const pokemons = [  
106   { id: 1, name: "Pikachu", type: "Electric", cp: 938 },  
107   { id: 2, name: "Bulbasaur", type: "Grass", cp: 850 },  
108   { id: 3, name: "Charmander", type: "Fire", cp: 880 }  
109 ];  
110  
111 console.log("Array of Pokemons:", pokemons);  
112 console.log("First Pokemon Name:", pokemons[0].name); // Pikachu  
113 console.log("Second Pokemon Type:", pokemons[1].type); // Grass
```

```
115 // วนลูปแสดงข้อมูล Array of Objects  
116 console.log("List of Pokemons:");  
117 for (const p of pokemons) {  
118   console.log(`- #${p.id} ${p.name} (Type: ${p.type}) Max CP: ${p.cp}`);  
119 }
```

HTML Forms

- ใช้สำหรับเก็บข้อมูลที่ใช้กรอก

<form>

...

form elements

...

</form>

First name:
Last name:

index.html · inline styles omitted

```
15 <form action="index.html" id="myForm" name="myForm">
18 First name:<br>
19 <input type="text" name="firstname" required><br>
20 Last name:<br>
21 <input type="text" name="lastname"><br>
24 </form>
```

Forms: Submit Button

- `<input type="submit">` ใช้สร้างปุ่มสำหรับส่งข้อมูลในฟอร์ม

index.html · inline styles omitted

```
15 <form action="index.html" id="myForm" name="myForm">
18 First name:<br>
19 <input type="text" name="firstname" required><br>
20 Last name:<br>
21 <input type="text" name="lastname"><br>
22 <input type="submit" value="Submit Form">
24 </form>
```

Profile Form (Form Event Demo)

First name:

Last name:

Forms: Action Attribute

- action ระบุปลายทางเมื่อส่งฟอร์มตามปกติ
- ตัวอย่างนี้ใช้ preventDefault() จึงไม่เปลี่ยนหน้า

index.html · inline styles omitted

```
15 <form action="index.html" id="myForm" name="myForm">
16 <fieldset>
17 <legend><strong>Profile Form (Form Event Demo)</strong></legend>
18 First name:<br>
19 <input type="text" name="firstname" required><br>
20 Last name:<br>
21 <input type="text" name="lastname"><br>
22 <input type="submit" value="Submit Form">
23 </fieldset>
24 </form>
```

Forms: Name Attribute

- ช่องรับข้อมูลแต่ละช่องต้องมีแอตทริบิวต์ name เพื่อระบุชื่อข้อมูลที่ส่ง หากไม่มี name ข้อมูลของช่องนั้นจะไม่ถูกส่ง

index.html · inline styles omitted

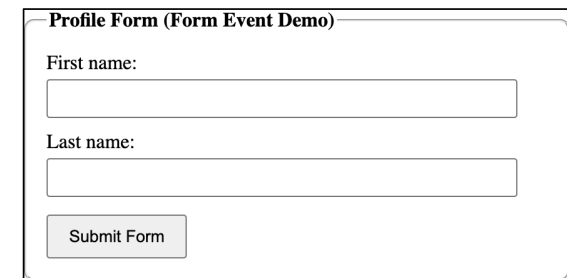
```
15 <form action="index.html" id="myForm" name="myForm">
18 First name:<br>
19 <input type="text" name="firstname" required><br>
20 Last name:<br>
21 <input type="text" name="lastname"><br>
22 <input type="submit" value="Submit Form">
24 </form>
```

Forms: Grouping Form Data

- `<fieldset>` ใช้รวมกลุ่มข้อมูลในฟอร์ม
- `<legend>` ใช้กำหนดคำอธิบายกลุ่มข้อมูลในฟอร์ม

index.html · inline styles omitted

```
15 <form action="index.html" id="myForm" name="myForm">
16 <fieldset>
17 <legend><strong>Profile Form (Form Event Demo)</strong></legend>
18 First name:<br>
19 <input type="text" name="firstname" required><br>
20 Last name:<br>
21 <input type="text" name="lastname"><br>
22 <input type="submit" value="Submit Form">
23 </fieldset>
24 </form>
```



Profile Form (Form Event Demo)

First name:

Last name:

JavaScript Form Validation

Attribute	Description
disabled	ปิดใช้งานช่องรับข้อมูล
max	กำหนดค่าสูงสุดที่ยอมรับได้
min	กำหนดค่าต่ำสุดที่ยอมรับได้
pattern	กำหนดรูปแบบที่ข้อมูลต้องตรงตามเงื่อนไข
required	กำหนดให้ผู้ใช้ต้องกรอกข้อมูลในช่องนี้
type	กำหนดชนิดของช่องรับข้อมูล

ตัวอย่าง

index.html · inline styles omitted

```
15 <form action="index.html" id="myForm" name="myForm">
16 <fieldset>
17 <legend><strong>Profile Form (Form Event Demo)</strong></legend>
18 First name:<br>
19 <input type="text" name="firstname" required><br>
20 Last name:<br>
21 <input type="text" name="lastname"><br>
22 <input type="submit" value="Submit Form">
23 </fieldset>
24 </form>
```

Profile Form (Form Event Demo)

First name:

Last name:

onsubmit Event

- เรียกฟังก์ชัน JavaScript เมื่อกดปุ่มส่งข้อมูล
- จากตัวอย่างเดิม
 - กำหนด id = “myForm” ให้กับฟอร์มใน HTML

```
main.js
125 window.onload = pageLoad;
126
127 function pageLoad() {
128     const form = document.getElementById("myForm");
129     if (form) {
130         form.onsubmit = myFormFunction;
131     }
132
133     loadStoredData();
134 }
135 }
```

Form elements Collection

- `document.forms` คือนคอลเลกชันของฟอร์มในหน้าเว็บ
- เราเข้าถึงข้อมูลในฟอร์มได้โดยใช้แอตทริบิวต์ `name`
- จากตัวอย่างเดิม
 - เพิ่ม `name = "myForm"` ให้กับฟอร์ม

```
main.js
137 function myFormFunction(event) {
139     event.preventDefault();
141     const firstnameInput = document.forms["myForm"]["firstname"].value;
142     const lastnameInput = document.forms["myForm"]["lastname"].value;
145     console.log("Firstname:", firstnameInput, "Lastname:", lastnameInput);
151 }
```

URLSearchParams()

- พารามิเตอร์ใน URL หรือพารามิเตอร์ในส่วนคำค้น ใช้ส่งข้อมูลสั้น ๆ ผ่าน URL ไปยังหน้าเว็บอื่นหรือเซิร์ฟเวอร์
- ตัวอย่างเมื่อเปิดหน้าเว็บพร้อมพารามิเตอร์ใน URL

```
window.location.search
```

```
?firstname=kejkaew&lastname=tha
```

URL parameters

URLSearchParams()

- เราใช้ API `URLSearchParams()` เพื่ออ่านพารามิเตอร์ใน URL ได้
- `URLSearchParams.get(parameter_name)` ใช้อ่านค่าแรกของพารามิเตอร์ที่ระบุชื่อ
- ตัวอย่างเช่น

main.js

```
161     const queryString = window.location.search;
162     if (queryString) {
163         const urlParams = new URLSearchParams(queryString);
164         const firstnameFromURL = urlParams.get('firstname');
165         const lastnameFromURL = urlParams.get('lastname');
166         console.log("Data from URL query:", firstnameFromURL, lastnameFromURL);
167     }
```

เมื่อ URL มี ?firstname=kejkaew&lastname=tha

ผลลัพธ์จาก console.log:

Data from URL query: kejkaew tha

LocalStorage

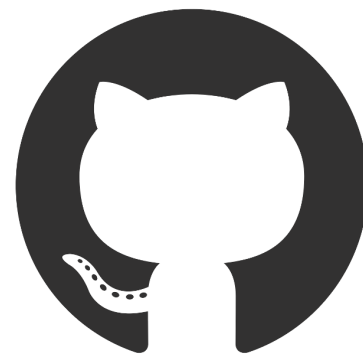
กลไกการจัดเก็บข้อมูลบนเว็บเบราว์เซอร์ของผู้ใช้ (Client-side Web Storage) ในรูปแบบ Key-Value Pair โดยข้อมูลที่บันทึกไว้จะ ไม่มีวันหมดอายุ (No Expiration Date) ข้อมูลจะยังคงอยู่แม้ผู้ใช้จะปิดแท็บ รีเฟรชหน้าเว็บ หรือปิดเบราว์เซอร์ไปแล้วก็ตาม จนกว่าจะถูกลบผ่านโค้ด JavaScript หรือผู้ใช้สั่งล้างแคชของเบราว์เซอร์

คุณสมบัติของ LocalStorage

- เก็บข้อมูลถาวร: ปิดเบราว์เซอร์แล้วเปิดใหม่ ข้อมูลก็ยังอยู่
- ความจุขนาดใหญ่: เก็บข้อมูลได้ประมาณ 5–10 MB ต่อโดเมน (มากกว่า Cookie ทั่วไปที่เก็บได้เพียง 4 KB)
- ทำงานเฉพาะฝั่ง Client: ข้อมูลจะไม่ถูกแนบไปกับทุก HTTP Request เหมือน Cookie ทำให้ประหยัดแบนด์วิดท์เครือข่าย
- เก็บบนเงื่อนไข Same-Origin Policy: ข้อมูลถูกแยกตามแต่ละ Origin (โปรโตคอล, โดเมน, และพอร์ตเดียวกันเท่านั้นจึงจะเข้าถึงได้)
- ชนิดข้อมูล: เก็บได้เฉพาะ ข้อความ (String) เท่านั้น (หากต้องการเก็บ Object หรือ Array ต้องแปลงเป็น JSON ก่อน)

LocalStorage ใช้งานผ่านอ็อบเจกต์ระดับ Global ในเบราว์เซอร์ได้ทันที โดยมี 4 เมธอดหลัก:

คำสั่ง	หน้าที่การทำงาน	ตัวอย่างไวยากรณ์
setItem(key, value)	บันทึกข้อมูลลง Storage	localStorage.setItem('username', 'somchai');
getItem(key)	ดึงข้อมูลออกมาใช้งานตามชื่อ Key	const user = localStorage.getItem('username');
removeItem(key)	ลบข้อมูลเฉพาะ Key ที่ระบุ	localStorage.removeItem('username');
clear()	ล้างข้อมูลทั้งหมดใน Storage ของโดเมนนั้น	localStorage.clear();



GitHub

เราจะใช้เผยแพร่หน้าเว็บเพื่อส่งแฟ้มสะสมผลงาน

What is GitHub

- เซิร์ฟเวอร์สำหรับให้บริการ Git
- พัฒนาต่อยอดจาก Git ซึ่งเป็นระบบควบคุมเวอร์ชัน
 - Git เป็นเครื่องมือที่ใช้งานผ่านบรรทัดคำสั่ง
 - GitHub มีส่วนติดต่อแบบกราฟิกที่ใช้งานผ่านเว็บ
- เก็บประวัติการเปลี่ยนแปลงของไฟล์ในโครงการ
- สามารถสร้างเว็บไซต์สำหรับโครงการได้

คำที่ควรรู้ใน GitHub

- Repository = แหล่งเก็บไฟล์และประวัติของโครงการที่ใช้ Git
- Clone = คัดลอกแหล่งเก็บโครงการจากเซิร์ฟเวอร์ระยะไกลลงเครื่อง
- Fork = สร้างสำเนาแหล่งเก็บโครงการไว้ในบัญชีของเรา
- Commit = บันทึกชุดการเปลี่ยนแปลงไว้ในประวัติเวอร์ชัน
- Pull request = คำขอให้ตรวจสอบและรวมการเปลี่ยนแปลงเข้ากับโครงการ
- Conflict = การแก้ไขส่วนเดียวกันที่ระบบไม่สามารถรวมให้อัตโนมัติได้
- Merge Commit = การบันทึกผลการรวมประวัติจากหลายสายการพัฒนา
- Fetch = ดึงข้อมูลและประวัติล่าสุดจากเซิร์ฟเวอร์ระยะไกล

ตัวอย่าง

- สมัครสมาชิก GitHub
- สร้าง repository

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?

[Import a repository.](#)

Owner



Repository name *

/ mdt-test1 ✓

Great repository names are short and memorable. Need inspiration? How about **vigilant-carnival**?

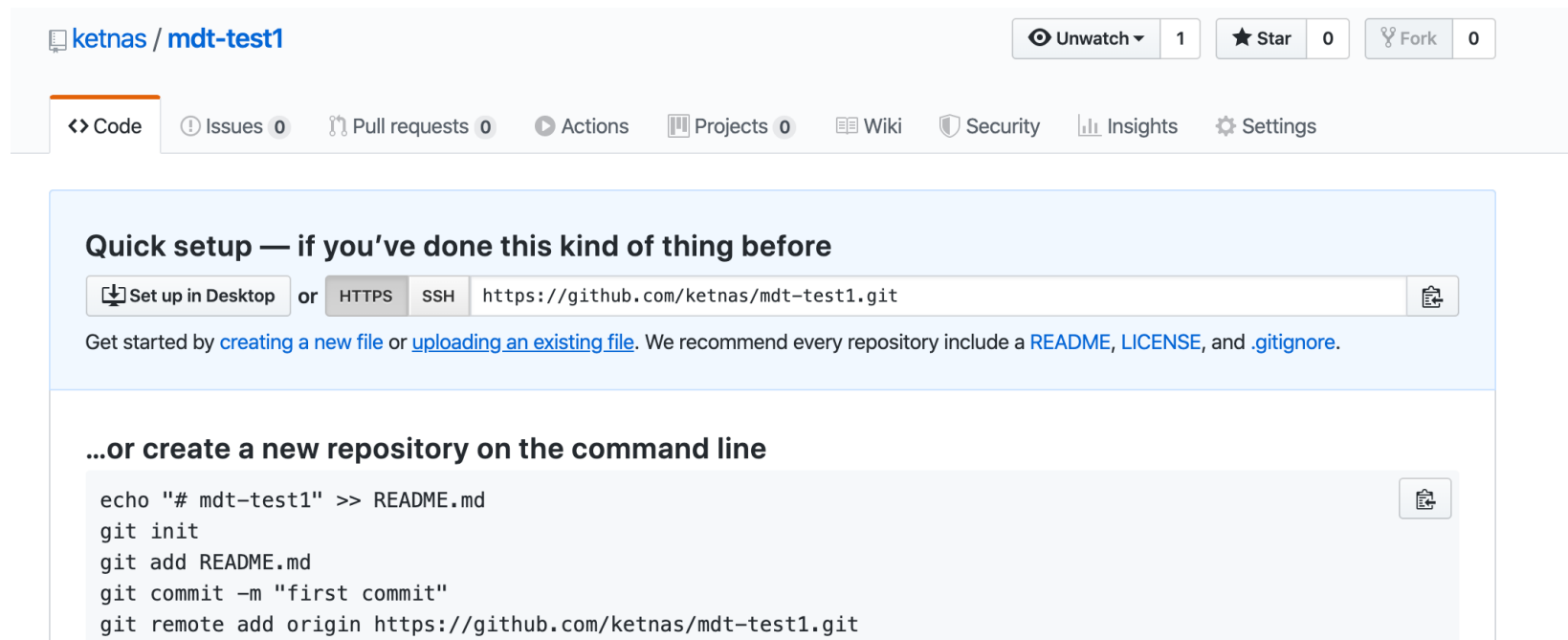
Description (optional)



Public

Anyone can see this repository. You choose who can commit.

■ Click at uploading an existing file



ketnas / mdt-test1

Unwatch 1 Star 0 Fork 0

<> Code Issues 0 Pull requests 0 Actions Projects 0 Wiki Security Insights Settings

Quick setup — if you've done this kind of thing before

 Set up in Desktop or ☐ HTTPS ☐ SSH

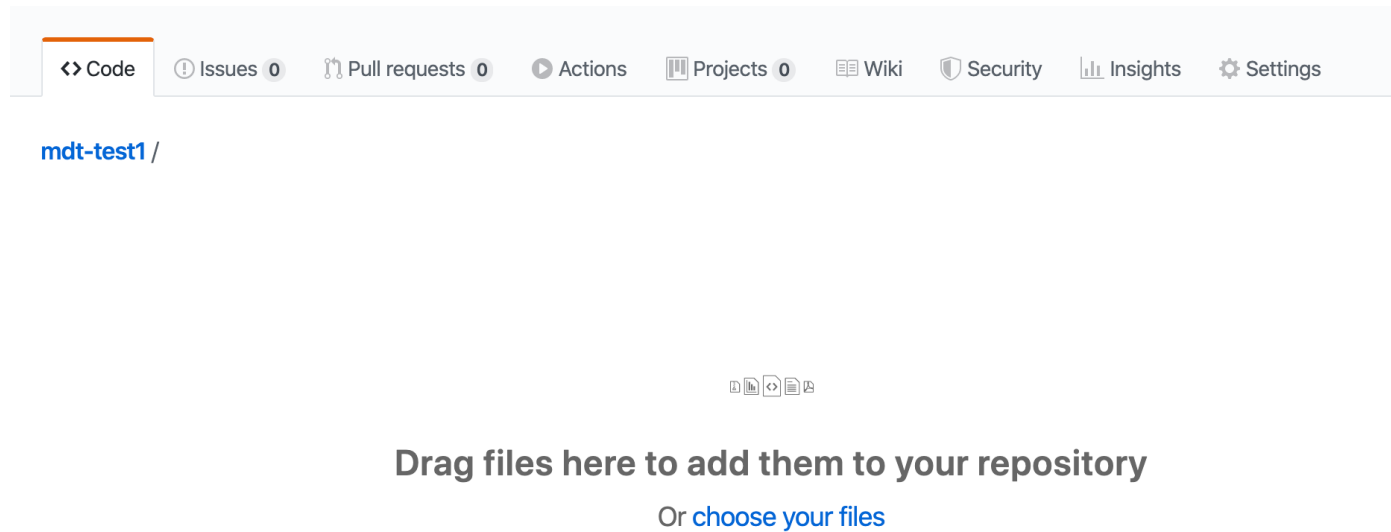
Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

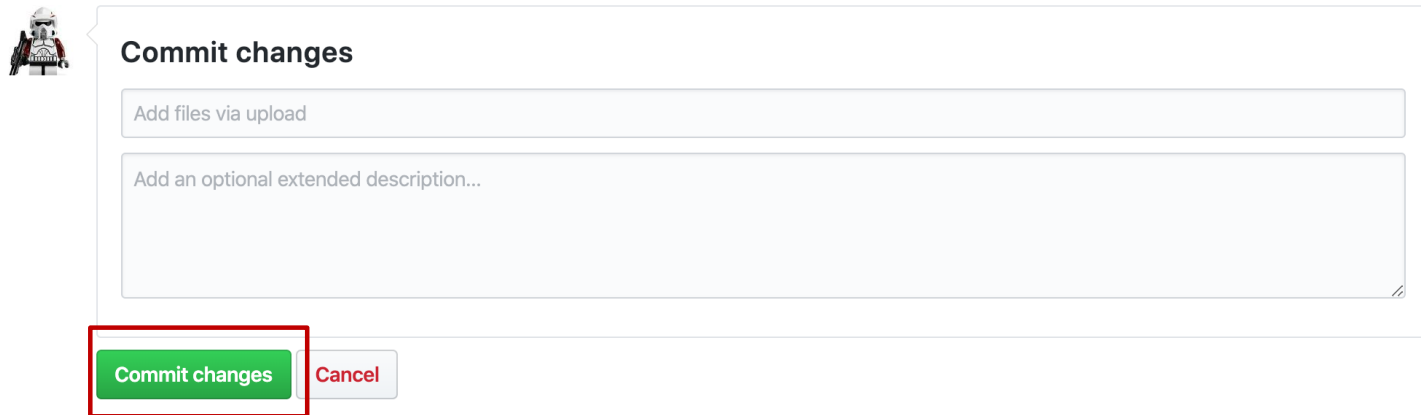
```
echo "# mdt-test1" >> README.md
git init
git add README.md
git commit -m "first commit"
git remote add origin https://github.com/ketnas/mdt-test1.git
```



■ ลาก index.html file มาวาง



■ กด commit



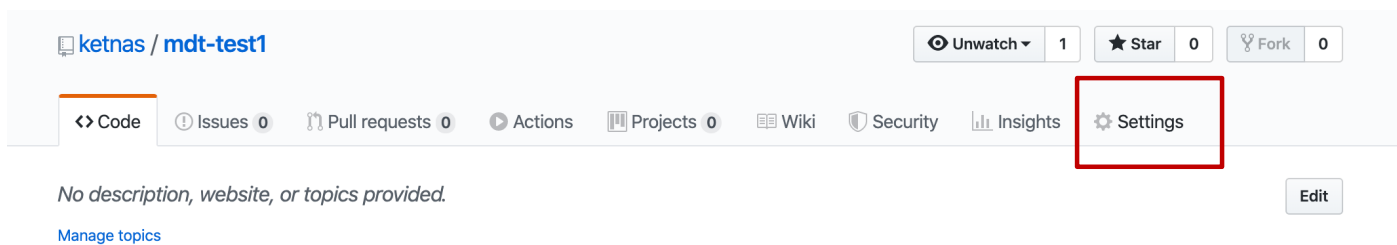
Commit changes

Add files via upload

Add an optional extended description...

Commit changes Cancel

■ กด setting



ketnas / mdt-test1

Unwatch 1 Star 0 Fork 0

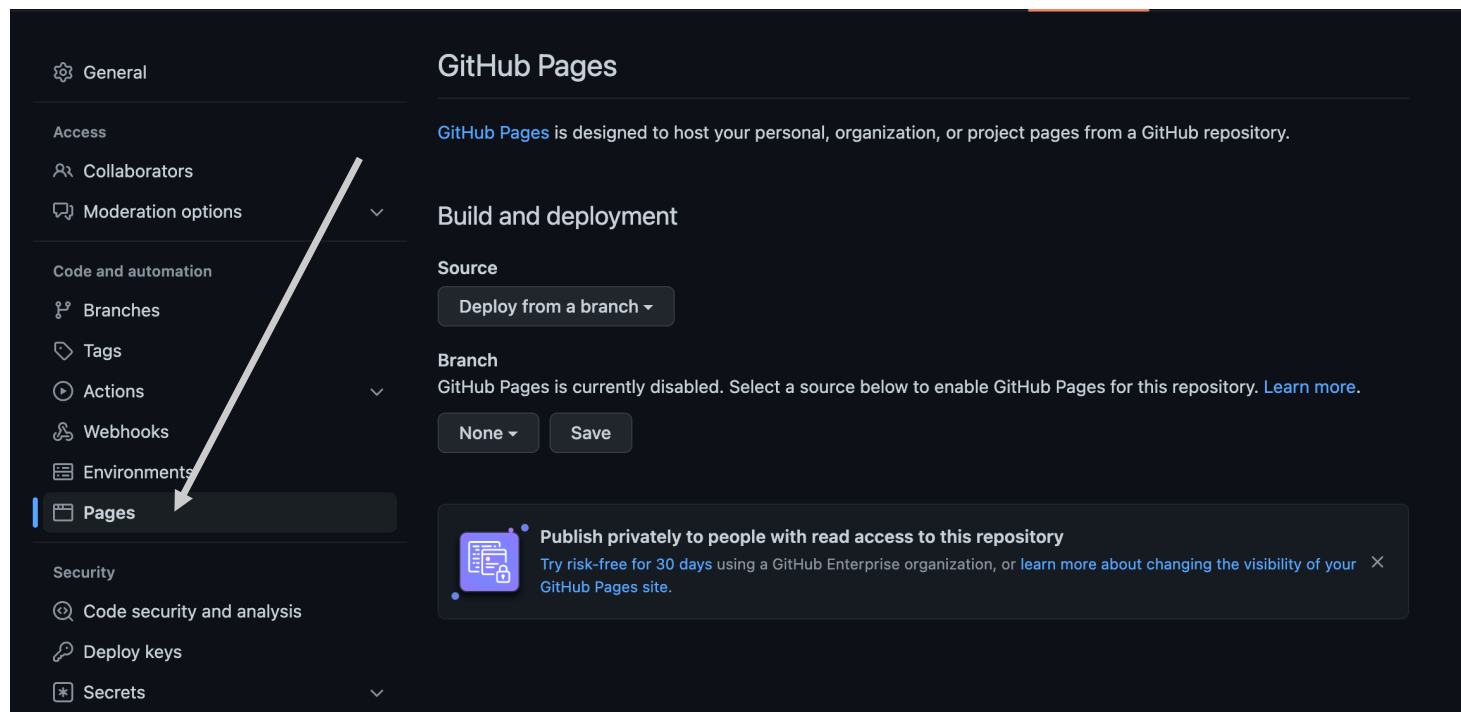
<> Code Issues 0 Pull requests 0 Actions Projects 0 Wiki Security Insights **Settings**

No description, website, or topics provided.

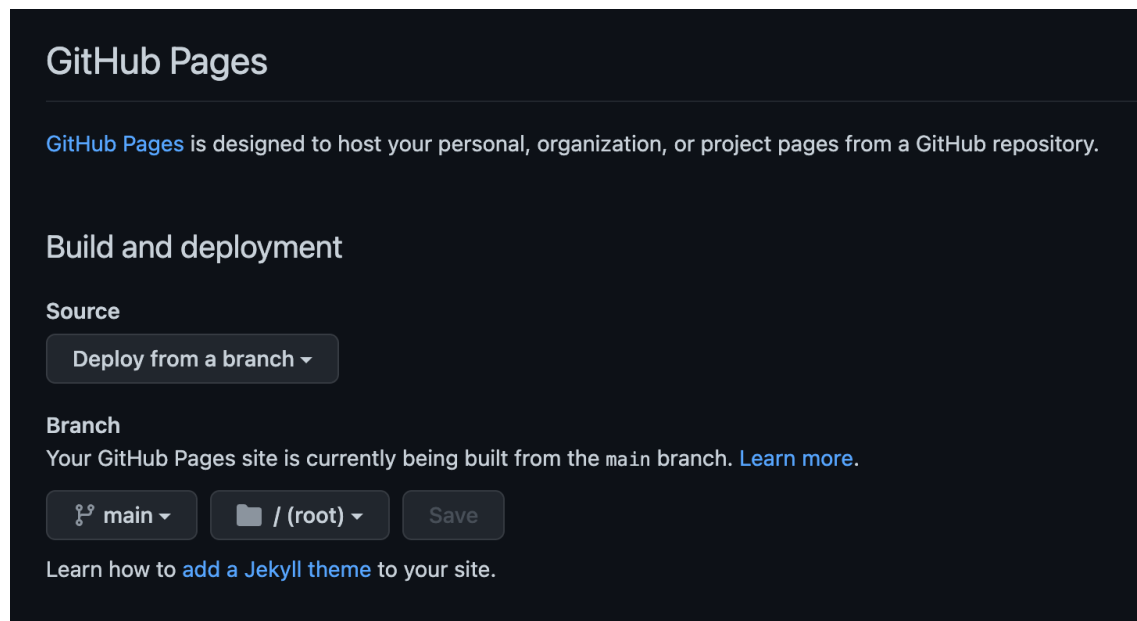
Manage topics

Edit

■ ไปที่ตรง Pages



- ที่ source เลือก Deploy from a branch และเลือก main Branch และกด save
- จะได้ URL เป็นชื่อ username ของเรา เช่น ketnas.github.io และตามด้วยชื่อ repository เช่น <https://ketnas.github.io/mdt-test1/>



Assignment 6: register and login

- ดาวน์โหลดไฟล์ assignment6.zip
- สร้างระบบลงทะเบียนและเข้าสู่ระบบตามรายละเอียดต่อไปนี้
 - หน้าลงทะเบียนต้องตรวจสอบว่าผู้ใช้กรอกข้อมูลครบทุกช่อง หากไม่ครบ ให้คงอยู่ที่หน้าลงทะเบียน
 - ตรวจสอบว่ารหัสผ่านกับช่องยืนยันรหัสผ่านตรงกัน หากไม่ตรงกัน ให้คงอยู่ที่หน้าลงทะเบียน
 - หน้าเข้าสู่ระบบต้องตรวจสอบว่าชื่อผู้ใช้และรหัสผ่านตรงกับข้อมูลที่ลงทะเบียนไว้ หากไม่ตรง ให้แสดง alert เพื่อให้กรอกใหม่
- การบ้าน: ตกแต่งหน้าลงทะเบียนและหน้าเข้าสู่ระบบให้สวยงาม

World frequency: b... https://app.firebaseio.com/ Google Gemini Firebase console All bookmarks

Register

* require

First name*: Last name*:

Gender*: ☐ Male ☐ Female ☐ Other

Birthday*:

Email address*:

Username*:

Password*:

Retype Password*:

Week 6: Pop Quiz

- ตอบคำถาม 8 ข้อแบบออนไลน์
- มีเวลา 15 วินาทีในการตอบแต่ละข้อ
- <https://ketnas.github.io/popQuiz/>

